MORCE: A Multi-Order Rule-Based Classifier Ensemble for Improving RACER Classification Accuracy

Kholood Khanaferi[a], Alireza Basiri[a,*], Mahroo Noohi[a]

[a]*Department of Electrical and Computer Engineering, Isfahan University of Technology, Isfahan, 84156-83111, Iran*

Abstract

Classification is one of the most widely studied tasks in machine learning and data mining, where the goal is to predict class labels for previously unseen data instances. Rule-based classifiers such as RACER learn classification rules from training data, but RACER's output can change noticeably depending on the order in which training instances are presented—different orderings of the same data can produce different rule sets and cause fluctuations in accuracy. This sensitivity, while often seen as a limitation, is exploited in this work as a source of ensemble diversity. The proposed method, MORCE (Multi-Ordering Rule-based Classifier Ensemble), shuffles the training data into multiple random orderings, trains an independent RACER classifier on each ordering, and combines the predictions of the resulting classifiers using several aggregation. Experimental results on 23 benchmark datasets demonstrate that all aggregation strategies significantly improve over the single-RACER baseline. These results confirm that selectively aggregating

*Corresponding author
Email address: basiri@iut.ac.ir (Alireza Basiri)

the predictions of the most accurate models reduces variance and enhances classification performance.
Keywords: Rule-based classification, RACER, Ensemble learning, Data-ordering sensitivity, Classifier aggregation, Stacking

---

1. Introduction

Machine learning has become a central tool in modern data analysis, aiming to discover patterns in data and use them to make predictions or decisions. Classification is one of the most fundamental tasks in machine learning and data mining, aiming to assign a discrete class label to each unseen instance based on labeled training examples. Over the years, a wide range of classification algorithms have been developed, such as decision trees [1], k-nearest neighbors [2], naive Bayes [3], support vector machines [4], and neural networks [5], each offering different trade-offs between predictive accuracy, computational efficiency, and model interpretability [6]. Among the many classification approaches, rule-based classifiers have attracted particular attention because they produce transparent, human-readable decision rules that can be easily interpreted, validated, and adjusted by domain experts. Such transparency is crucial in sensitive domains where understanding the reasoning behind a prediction is as important as the prediction itself. Rule-based methods construct a set of if-then rules that partition the feature space, and use these rules to assign class labels to new instances. However, the effectiveness of a rule-based classifier strongly depends on how well its rules are learned and how robust they are to variations in the training data.

Over the years, a variety of algorithms have been developed for learn-

ing classification rules. Among the more recent ones, the Rule Aggregating Classifier (RACER) [7] takes a fundamentally different approach to rule construction. Unlike conventional rule learners that build rules incrementally, RACER treats each training instance as an initial rule and then iteratively merges and generalizes these rules to obtain a compact and effective rule set. Despite these strengths, RACER exhibits a notable limitation: its performance is sensitive to the order in which training instances are presented, and different orderings of the same training data can produce different final rule sets, causing fluctuations in classification accuracy and results that cannot be considered fully reliable.

Ensemble methods provide a principled approach to addressing this kind of instability. The core idea is to train multiple base classifiers—which may be of the same or different types—and combine their predictions, so that the errors of individual models are smoothed out and the overall decision becomes more reliable [8, 9, 10]. Well-known ensemble strategies such as Bagging [11], Boosting [12], and Random Forests [13] achieve this by introducing diversity among the base learners—through techniques like bootstrap sampling, sequential error correction, or random feature subsets. A central principle in ensemble learning is that when the individual classifiers produce sufficiently diverse and uncorrelated errors, their combination leads to significantly better performance than any single model alone.

Building on this principle, we propose an ensemble-based extension of RACER that turns its sensitivity to data ordering into a source of diversity. Our approach, MORCE (Multi-Ordering Rule-based Classifier Ensemble), creates multiple random orderings of the training data, trains a separate

RACER model on each ordering, and then combines their predictions using seven aggregation strategies, spanning both voting-based and stacking-based schemes. By integrating the outputs of these diverse models, MORCE reduces variance, stabilizes predictions, and improves classification performance compared to a single RACER baseline.

The rest of this paper is organized as follows. Section 2 reviews the related work. Section 3 provides background on the RACER algorithm, covering its preprocessing, rule representation, composition, and generalization phases. Section 4 presents the proposed approach, describing the effect of data ordering on RACER and the aggregation methods employed, including voting-based and stacking-based strategies. The experimental results are detailed in Section 5, and the conclusion is drawn along with future work in Section 6.

2. Related Work

Interpretable classification has long been studied through rule-based and tree-based learners, which provide human-readable decision structures in the form of *if-then* rules. In contrast to black-box models, these approaches make it possible to trace the rationale behind each prediction, which is especially important in domains where transparency and accountability are required. A rule typically consists of an antecedent, formed by a conjunction of attribute conditions, and a consequent, which denotes the predicted class label. Decision trees provide one of the most intuitive frameworks for such representations, since every path from the root to a leaf can be translated into a classification rule. Among the earliest and most influential tree-based

methods, ID3 [1] constructs decision trees by selecting attributes according to information gain. C4.5 [14] extends this idea by supporting continuous attributes, handling missing values, and applying pruning to improve generalization. CART [15] follows a similar recursive partitioning strategy but uses the Gini impurity criterion and binary splits. More recent tree-based methods such as ForestPA [16], Optimized Forest [17], LMT [18], and SPAARC [19] have introduced various improvements in accuracy and efficiency.

A second major family of interpretable models consists of rule-based algorithms that learn rules directly from data, without constructing an intermediate tree [20, 21]. CN2 [22] uses a beam-search strategy guided by an entropy-based evaluation function to discover useful rules. PART [23] builds partial decision trees repeatedly and extracts the best leaf as a rule. RIPPER [24] adopts a grow-and-prune procedure followed by global optimization, making it effective on noisy datasets. These algorithms follow the sequential covering paradigm, in which rules are learned one at a time and the instances covered by each rule are removed before the next one is induced. A different line of work avoids sequential rule induction altogether: MCRM [25], for instance, is a meta-heuristic rule learner that extracts the entire rule set simultaneously by formulating rule construction as a global optimization problem solved with the Artificial Bee Colony (ABC) algorithm[26], while also integrating multivariate discretization and implicit feature selection into the learning process, at the cost of higher computational overhead.

Another notable algorithm in the rule-based family is RACER [7], a classification algorithm that represents each training instance as an initial rule and iteratively merges and generalizes these rules to produce a compact and

effective classifier. Unlike conventional sequential covering methods, which learn one rule at a time and discard the instances it covers, RACER begins by treating all training instances as initial rules and then combines them during learning, which gives it strong predictive capability. The rule-merging process, however, is itself carried out sequentially: the resulting rule set depends on the order in which the initial rules are considered for combination, since different orderings lead the merging process along different paths and yield different final rule sets. Thus, even though RACER does not belong to the sequential covering family, it remains sensitive to the order of the training data.

Since its introduction, several extensions have been proposed to improve different aspects of RACER's behavior. GRACER [27] introduces greedy pair selection during rule merging to improve rule quality. ROPAC [28] enhances initial rule quality through zero-bit frequency analysis. RACEkNN [29] combines RACER's rule partitioning with k-nearest neighbor classification to improve accuracy. RACE-RANK [30] applies a PageRank-inspired scheme to prioritize initial rules before merging.Despite the diversity of these extensions, none of them adopts an ensemble-based approach to improve the classification performance of RACER. In a related direction, Bergamin et al. [31] proposed a framework based on Bayes point aggregation that combines multiple rule sets into more stable predictions; their work, however, targets sequential covering learners in general rather than RACER. This motivates applying ensemble learning to RACER, a direction that, to the best of our knowledge, no prior study has investigated.

3. RACER

RACER is a rule-based classification algorithm designed to improve predictive accuracy on unseen instances. It begins by treating each training instance as a primitive rule, then merges and generalizes these rules to yield a compact and effective classifier. The algorithm proceeds through several sequential phases, beginning with preprocessing and rule representation, followed by rule composition and fitness evaluation, and concluding with rule generalization and classifier construction.

3.1. Preprocessing

Since RACER is designed to handle categorical data, a discretization step is required to transform numerical attributes into discrete form. This transformation relies on entropy minimization to determine optimal splitting thresholds.

Given a numerical attribute A, the algorithm first arranges its values in ascending order. Candidate split points are then generated by computing the midpoint between each pair of consecutive values:

$$MidPoint_i = \frac{a_i + a_{i+1}}{2}, \quad i = 1, 2, \ldots, N \tag{1}$$

To evaluate each candidate, the dataset D is partitioned into two subsets: D_1 containing instances where $A \leq MidPoint_i$, and D_2 containing the rest. The quality of this split is measured by the weighted entropy:

$$Info_{A_i}(D) = \sum_{j=1}^{2} \frac{|D_j|}{|D|} \times Info(D_j) \tag{2}$$

The entropy of each subset is calculated as:

$$Info(D) = -\sum_{i=1}^{m} p_i \times \log_2(p_i) \tag{3}$$

where $p_i = |C_{i,D}|/|D|$ denotes the proportion of instances belonging to class C_i, and m represents the total number of classes. The candidate split point that produces the lowest weighted entropy is selected for discretization. This process converts the continuous attribute into a categorical feature.

3.2. Rule Representation

RACER adopts a binary encoding scheme to represent classification rules. Each rule consists of two parts: the left-hand side encodes feature conditions, while the right-hand side specifies the predicted class. For every categorical feature, each possible value is assigned a dedicated bit. A bit set to 1 indicates that the rule accepts instances having the corresponding feature value, while a bit set to 0 means that value does not participate in the rule condition. This design ensures that all rules share the same fixed length. For further clarity, consider the following domain features F_1 through F_5, and F_c as the class label:

Domain features: F_1, F_2, F_3, F_4, F_5

Class label: F_c

$F_1 \in \{Toyota, BMW, Honda\} \Rightarrow 3$ bits

$F_2 \in \{gasoline, electric\} \Rightarrow 2$ bits

$F_3 \in \{budget, mid, luxury\} \Rightarrow 3$ bits

$F_4 \in \{low, medium, high\} \Rightarrow 3$ bits

$F_5 \in \{new, used\} \Rightarrow 2$ bits

$F_c \in \{Buy, Pass\} \Rightarrow 2$ bits

Each rule consists of 15 bits: 3 bits for F_1, 2 bits for F_2, 3 bits for F_3, 3 bits for F_4, 2 bits for F_5, and 2 bits for F_c. For instance, given $F_1 = Toyota$, $F_2 = electric$, $F_3 = mid$, $F_4 = low$, and $F_5 = new$, the resulting initial rule is:

F_1	F_2	F_3	F_4	F_5	F_c
100	01	010	100	10	$\rightarrow 10$

Consider a more general rule:

F_1	F_2	F_3	F_4	F_5	F_c
100	11	011	100	10	$\rightarrow 10$

The interpretation is: if (F_1 = Toyota) and (F_2 = gasoline or electric) and (F_3 = budget or mid) and (F_4 = low) and (F_5 = new), the outcome is Buy. Since $F_2 = 11$ covers all possible values, it becomes redundant and the rule simplifies to: if (F_1 = Toyota) and (F_3 = budget or mid) and (F_4 = low) and (F_5 = new) then Buy.

3.3. Creating Initial Rules

Based on the binary representation, every training instance directly becomes an initial rule. With N training samples, we obtain N initial rules at no extra cost. Before showing an example, the complete binary mapping for each feature is presented below:

Table 1: Binary encoding of all feature values

Feature	F_1 (Brand)			F_2 (Fuel Type)		F_3 (Price)			F_4 (Mileage)			F_5 (Condition)		F_c (Decision)	
Value	Toyota	BMW	Honda	gasoline	electric	budget	mid	luxury	low	medium	high	new	used	Buy	Pass
Code	100	010	001	10	01	100	010	001	100	010	001	10	01	10	01

Suppose the training set contains the following instance:

F_1 = Toyota, F_2 = electric, F_3 = mid, F_4 = low, F_5 = new, F_c = Buy

Based on the binary mapping above, this instance is converted into the following rule:

F_1	F_2	F_3	F_4	F_5	F_c
100	01	010	100	10	$\rightarrow$ 10
(Toyota)	(electric)	(mid)	(low)	(new)	(Buy)

This rule states: If (F_1 = Toyota) and (F_2 = electric) and (F_3 = mid) and (F_4 = low) and (F_5 = new) then the decision is Buy.

3.4. Rule Composition

Although initial rules perform perfectly on training data, their high specificity limits generalization to new instances. Each initial rule covers only one training sample, making it ineffective for unseen data. RACER overcomes this by merging initial rules to form generalized rules with broader coverage.

The merging operation applies bitwise OR between two same-class rules. The result is a more general rule that accepts instances matched by either input rule. Importantly, composition occurs only if the new rule has higher fitness than both originals. Consider the following two rules:

	F_1	F_2	F_3	F_4	F_5	F_c
Rule 1:	100	01	010	100	10	$\rightarrow$ 10
Rule 2:	001	01	010	100	10	$\rightarrow$ 10

The bitwise OR operation yields:

	F_1	F_2	F_3	F_4	F_5	F_c
Combined:	101	01	010	100	10	$\rightarrow 10$

Interpretation: If (F_1 = Toyota or Honda) and (F_2 = electric) and (F_3 = mid) and (F_4 = low) and (F_5 = new) then Buy.

The fitness evaluation in RACER relies on two fundamental criteria: accuracy and coverage. These metrics determine whether combining two rules produces a better outcome.

Accuracy measures the proportion of correctly classified instances among all instances covered by a rule:

$$accuracy(R_i) = \frac{n_{correct}}{n_{covers}} \tag{4}$$

where $n_{correct}$ is the number of training instances correctly classified by rule R_i, and n_{covers} is the total number of instances that match the rule.

Coverage indicates how many training instances a rule can handle relative to the entire training set:

$$coverage(R_i) = \frac{n_{covers}}{|D_{tr}|} \tag{5}$$

where $|D_{tr}|$ represents the size of the training dataset.

The overall fitness combines both metrics using weighted coefficients:

$$Fitness(R_i) = \alpha \times accuracy(R_i) + \beta \times coverage(R_i) \tag{6}$$

subject to: $\alpha + \beta = 1$ and $\alpha, \beta > 0$. The parameters α and β control the trade-off between accuracy and coverage. Higher α prioritizes precise classification, while higher β favors rules that cover more instances. During the composition phase, RACER compares the fitness of the combined rule

with both original rules. If the combined rule achieves a higher fitness value, the two initial rules are removed and replaced by the new composed rule. Otherwise, both original rules remain unchanged in the rule set.

3.5. Rule Generalization and Classifier Construction

After the composition phase, RACER attempts to generalize the learned rules further. The generalization process identifies all bits with a value of 0 in a rule and tries flipping each one to 1. Changing a 0 to 1 makes the rule more general because the rule will then match additional feature values. For each 0-bit, RACER temporarily changes it to 1 and evaluates the new fitness. If the fitness improves, the change is kept; otherwise, the bit reverts to 0. This process repeats for every 0-bit in the rule.

After the generalization phase, RACER orders all rules in descending order of their fitness values. To classify a new instance, the algorithm scans this ordered list from highest to lowest fitness, and the first rule whose conditions are satisfied by the instance is used to assign the class label.

4. Proposed Approach

As discussed earlier, RACER is sensitive to the order in which the training instances are presented, and the reason becomes clear from the way it builds its rule set. Given N training instances, RACER first turns each instance directly into an initial rule, producing n initial rules whose order is exactly the order of the training data. In the merging phase, these rules are then processed sequentially: a pair of same-class rules is merged whenever the combined rule improves fitness, and once a rule is absorbed into a more general one it is no longer considered for further merging. Because the rules are

examined in the order inherited from the training data, presenting the same instances in a different order changes which pairs are merged first, drives the merging process along a different path, and ultimately yields a different final rule set. Since classification of the test data relies entirely on this learned rule set, different orderings of the same training data can assign different labels to the same test instances, so the reported accuracy varies from one ordering to another. This makes the predictions of a single RACER model unstable and difficult to rely on. To turn this behavior into an advantage, we train n independent RACER classifiers, each on a different random ordering of the same training data, so that each classifier produces its own distinct rule set. At test time, every classifier uses its own rule set to generate its own predictions for the test instances, and these per-model predictions are then aggregated across all classifiers to produce the final labels. Combining the predictions in this way yields more stable, lower-variance decisions and improves classification performance on unseen data. In this work, RACER serves as the base classifier, and the goal is to improve its classification accuracy through this ensemble-based approach. The complete algorithm and the aggregation strategies are described in detail in the following subsections, and Figure 1 illustrates the overall architecture of the proposed method.

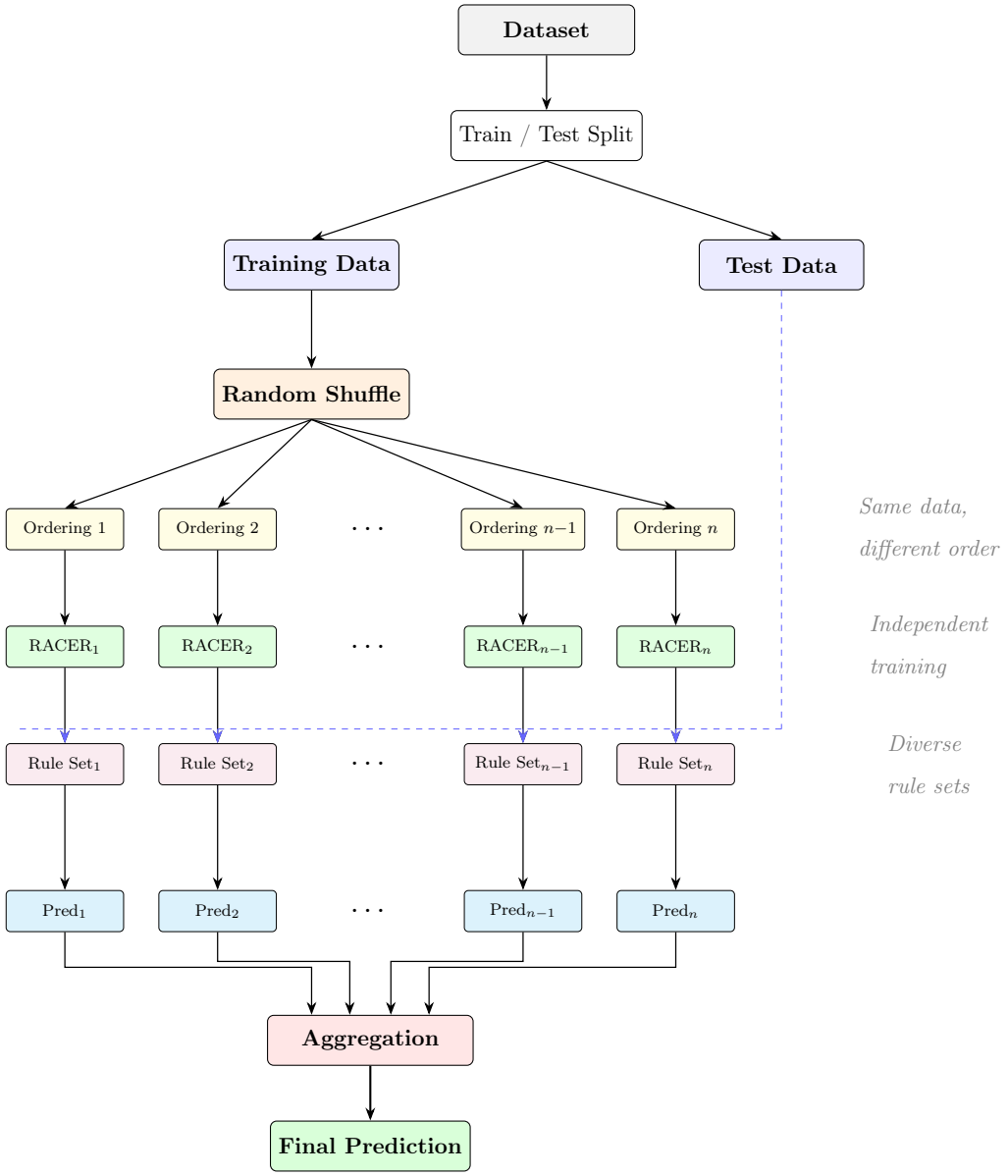


Figure 1: Overview of the proposed ensemble approach based on n independent RACER classifiers trained on different random orderings of the training data.

4.1. Effect of Data Ordering on RACER

To make this behavior concrete, Figure 2 illustrates it on a small example of eight training instances. Each of the eight instances shown in the table is encoded directly as one initial rule, and these rules are then presented to RACER under three different random orderings. Inside each RACER classifier, the algorithm automatically separates the rules by class and applies the merging and generalization procedure described in Section 3 to each class on its own, composing rules only with others of the same label; in this example, the Buy rules and the Pass rules are therefore processed separately, and each class yields its own part of the final rule set. Although every ordering starts from exactly the same set of rules, the order in which the rules are visited determines which same-class pairs are merged first; because different orderings bring different pairs together, the composition paths diverge and lead to distinct final rule sets.

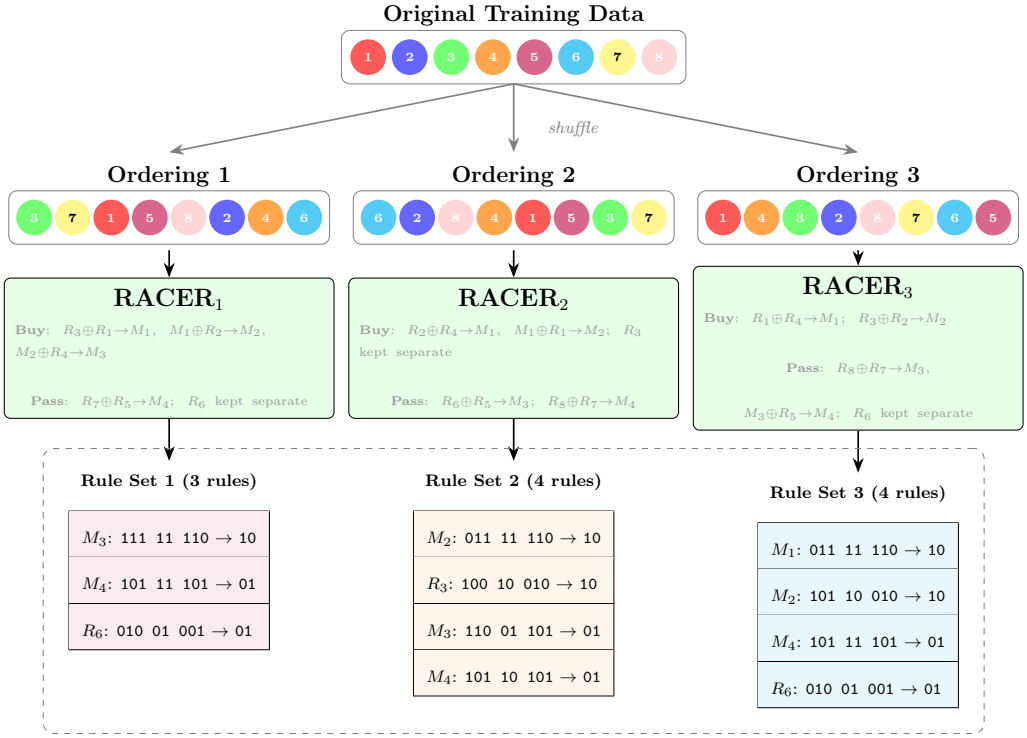
As the figure shows, when the ordering changes it is not only the model's predictions that differ: we obtain different rule sets, differing both in the number of rules and in the rules themselves, as well as in how often a given rule recurs across orderings. Some rules appear independently under several orderings, and are therefore likely to capture a genuine pattern in the data, while others appear under a single ordering only, and are therefore likely to be accidental. This variation among the rule sets is the central insight of the present work: what appears as instability in a single RACER model is, when harnessed across many orderings, exactly the diversity an ensemble needs.

Training Data (8 instances) and Their Initial Rules

#	F_1 (Brand)	F_2 (Fuel)	F_3 (Price)	Class	Initial Rule
1	B (010)	X (10)	P (100)	Buy	010 10 100 → 10
2	C (001)	X (10)	Q (010)	Buy	001 10 010 → 10
3	A (100)	X (10)	Q (010)	Buy	100 10 010 → 10
4	C (001)	Y (01)	Q (010)	Buy	001 01 010 → 10
5	A (100)	Y (01)	P (100)	Pass	100 01 100 → 01
6	B (010)	Y (01)	R (001)	Pass	010 01 001 → 01
7	C (001)	X (10)	R (001)	Pass	001 10 001 → 01
8	A (100)	X (10)	P (100)	Pass	100 10 100 → 01

Domains: $F_1 \in \{A{=}100, B{=}010, C{=}001\}$, $F_2 \in \{X{=}10, Y{=}01\}$, $F_3 \in \{P{=}100, Q{=}010, R{=}001\}$, *Class* $\in \{Buy{=}\ 10,\ Pass{=}\ 01\}$

Each training instance maps directly to one initial rule.



Same instances, different order ⇒ Different composition sequence ⇒ Different rule sets

Figure 2: Effect of data ordering on RACER.

4.2. Proposed Ensemble Approach

In the proposed approach, the training data is first shuffled into n different random orderings. Each ordering is then fed as input to an independent instance of the RACER algorithm, so that each classifier independently extracts its own rule set. Once a classifier's rule set has been built, the model is applied back to the training data to obtain its predictions there, and the resulting training accuracy is stored for later use; this accuracy is subsequently used by the weighted and top-k aggregation methods to determine the contribution of each model to the final decision. After all n classifiers have been trained and their training accuracies recorded, the focus shifts to classifying the test data: each classifier independently predicts the class labels for all m test instances at once using its own rule set. This produces an $n \times m$ prediction matrix, where each row corresponds to one RACER classifier and contains its predictions for all m test instances, and each column corresponds to a single test instance and gathers the predictions of all n classifiers for it. The final label of each test instance is then obtained by combining the n predictions in its corresponding column using an aggregation method. Figure 3 illustrates this process. In the following, each aggregation method is described in detail.

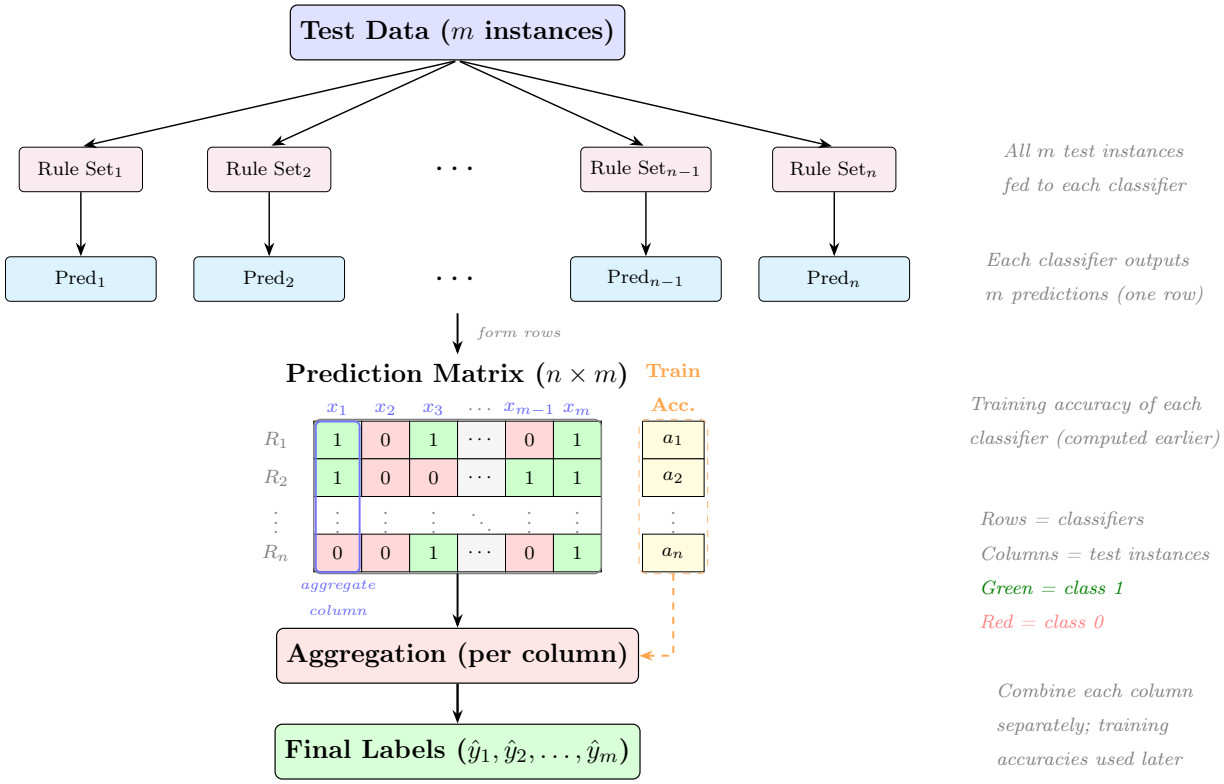


Figure 3: Test classification process.

4.3. Aggregation Methods

Once the n RACER classifiers have produced the $n \times m$ prediction matrix, together with the training accuracy of each classifier, the final class label of every test instance is determined by combining the n predictions in its corresponding column. In this work, several aggregation strategies are explored, which fall into two categories: voting-based methods and stacking-based methods. In voting-based methods, the predictions are combined directly through different voting schemes, whereas in stacking-based methods they are treated as input features for a second-level model that learns how to map them to the final label. All of these methods apply to both binary and

multi-class classification problems. Below, each category and its variants are described in detail.

4.4. Voting-based Aggregation

In voting-based methods, each classifier directly contributes to the final decision by casting a vote for a class label. The key difference among the variants lies in how many classifiers participate in the voting and whether their votes are equally weighted or not.

4.4.1. Majority Voting

In this method, every classifier casts one vote for the class it predicts, and the class with the most votes is chosen as the final label. All classifiers have equal weight, and no classifier takes priority over another; ties are broken by selecting one of the tied labels at random. The scheme applies directly to both binary and multi-class problems. As an example, suppose ten classifiers produce the predictions shown in Table 2 for a single test instance.

Table 2: Example predictions of ten RACER classifiers for a single test instance.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}	$\Rightarrow \hat{y} = 1$
Prediction	1	0	1	1	0	1	1	0	1	1	

Here, six classifiers predict class 1 and four predict class 0, so class 1 receives the majority of the votes and becomes the final prediction for this instance. The same procedure is repeated independently for every test instance to produce the predicted labels for the entire test set.

4.4.2. Top-k Voting

This method first sorts all n classifiers in descending order of their training accuracy — the accuracy computed in the previous section, where each classifier was applied to the training data — and then keeps only the top k classifiers. The final decision is made by majority voting among these k classifiers alone. The aim is to exclude the weaker classifiers so that the outcome is not degraded by their less reliable predictions. As an example, suppose ten classifiers obtain the training accuracies shown in Table 3.

Table 3: Example training accuracies of ten RACER classifiers ($n = 10$).

RACER Classifier	Training Accuracy
R_1	0.85
R_2	0.72
R_3	0.91
R_4	0.68
R_5	0.88
R_6	0.79
R_7	0.93
R_8	0.74
R_9	0.90
R_{10}	0.82

After sorting by training accuracy in descending order and selecting the top $k = 5$ classifiers, Table 4 shows their predictions for a single test instance together with the resulting output.

Table 4: Top-k voting example ($k = 5$).

Classifier	Training Acc.	Prediction	Selected
R_7	0.93	1	✓
R_3	0.91	1	✓
R_9	0.90	0	✓
R_5	0.88	1	✓
R_1	0.85	1	✓
R_{10}	0.82	1	—
R_6	0.79	0	—
R_8	0.74	0	—
R_2	0.72	0	—
R_4	0.68	0	—
Top-k votes: class $1 = 4$, class $0 = 1$		$\Rightarrow\ \hat{y} = 1$	

4.4.3. Weighted Voting

In this method, each RACER classifier is assigned a weight equal to its training accuracy, computed as described in the previous section. For a given test instance, the weight of each classifier is added to the class it has predicted, and the class receiving the highest total weight is selected as the final output. Formally, the final prediction for a test instance x is

$$\hat{y}(x) = \arg\max_{c \in C} \sum_{i=1}^{n} a_i \cdot \mathbb{I}[p_i(x) = c] \tag{7}$$

where a_i is the training accuracy of classifier R_i, $p_i(x)$ is its prediction for x, and $\mathbb{I}[\cdot]$ is the indicator function. Unlike majority voting, where every classi-

fier contributes equally, a more accurate classifier here exerts proportionally greater influence on the outcome. As an example, consider the training accuracies and predictions of ten RACER classifiers for a single test instance, shown in Table 5.

Table 5: Example training accuracies and predictions of ten RACER classifiers for a single test instance.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
Training Acc.	0.85	0.78	0.92	0.80	0.75	0.88	0.83	0.70	0.90	0.86
Prediction	1	0	1	1	0	1	0	0	1	1

The total weight of each class is obtained by summing the training accuracies of the classifiers that predicted it, as shown in Table 6.

Table 6: Weighted vote computation for the example test instance.

Class	Contributing Classifiers	Total Weight
0	R_2, R_5, R_7, R_8	$0.78 + 0.75 + 0.83 + 0.70 = 3.06$
1	R_1, R_3, R_4, R_6, R_9, R_{10}	$0.85 + 0.92 + 0.80 + 0.88 + 0.90 + 0.86 = 5.21$

Since $W(1) = 5.21 > W(0) = 3.06$, the final prediction for this instance is class 1. The same procedure is applied independently to every test instance to obtain the predicted labels for the entire test set.

4.4.4. Top-k Weighted Voting

This method combines the two preceding approaches. The n classifiers are first sorted by their training accuracy and only the top k are retained;

weighted voting is then performed among these k classifiers alone, using their training accuracies as weights:

$$\hat{y}(x) = \arg\max_{c \in C} \sum_{i \in \mathcal{S}_k} a_i \cdot \mathbb{I}[p_i(x) = c] \tag{8}$$

where $\mathcal{S}_k$ denotes the index set of the k classifiers with the highest training accuracy. Weaker classifiers are thus excluded entirely, while among the retained ones the stronger are given greater importance. As an example, using the same training accuracies as before, consider a single test instance whose predictions are shown in Table 7.

Table 7: Example training accuracies and predictions of ten RACER classifiers for a single test instance.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
Training Acc.	0.85	0.78	0.92	0.80	0.75	0.88	0.83	0.70	0.90	0.86
Prediction	1	0	0	1	0	1	0	0	0	1

The five classifiers with the highest training accuracy are R_3 (0.92), R_9 (0.90), R_6 (0.88), R_{10} (0.86), and R_1 (0.85). Weighted voting is then performed among these five alone, as shown in Table 8.

Table 8: Top-k weighted vote computation for the example test instance ($k = 5$).

Class	Contributing Classifiers	Total Weight
0	R_3, R_9	$0.92 + 0.90 = 1.82$
1	R_1, R_6, R_{10}	$0.85 + 0.88 + 0.86 = 2.59$

Since $W(1) = 2.59 > W(0) = 1.82$, the final prediction is class 1. Note

that weighted voting over all ten classifiers would have favoured class 0 in this example, since the excluded classifiers predominantly predict that class; restricting the vote to the most accurate classifiers therefore changes the outcome. The same procedure is carried out for every test instance to produce the final classification for the entire test set.

4.5. Stacking-based Aggregation

In this approach, the predictions already obtained from each base classifier are used as input features for a second-level model (meta-model) that learns how best to combine them[32]. For the meta-model,the prediction matrix is organized so that each row corresponds to a single instance and each column to one RACER classifier. The meta-model is trained on the predictions that the n RACER classifiers make on the training data: each training instance forms one row of n features — the labels assigned to it by the n classifiers — and the true label of that instance is used as the target. From these examples the meta-model learns an appropriate combination pattern. At test time, the predictions of the n classifiers on each test instance are arranged in the same way and passed to the trained meta-model, whose output is taken as the final label of that test instance, as illustrated in Figure 4.

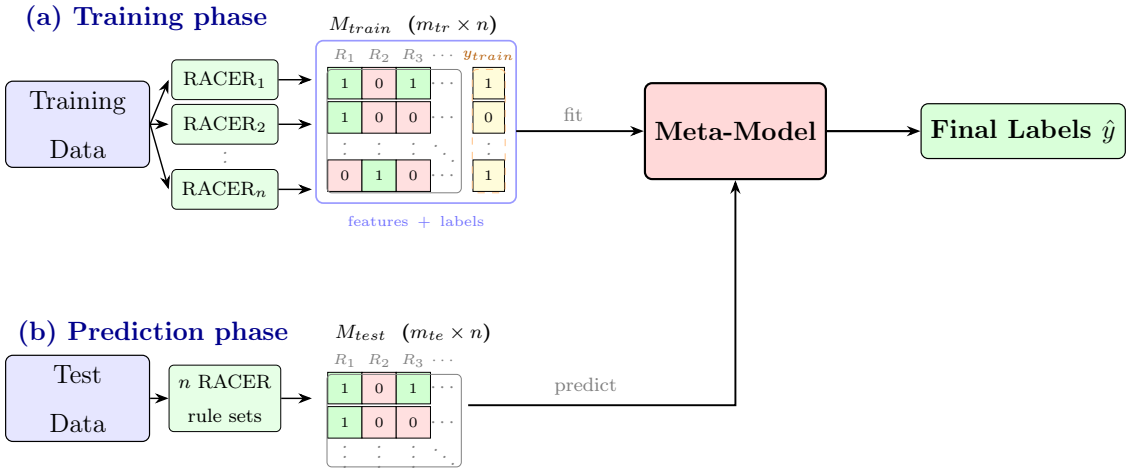


Figure 4: Stacking-based aggregation: the n RACER classifiers' predictions on the training data, together with the true labels, fit a single meta-model, which then maps their predictions on the test data to the final labels.

4.5.1. Stacking with Logistic Regression

Here a logistic regression model serves as the meta-model, learning how to combine the predictions of the base classifiers. In effect, it learns an appropriate weight for each classifier and produces the final decision accordingly. For example, suppose the 10 RACER instances produce the predictions shown in Table 9 for five training instances.

Table 9: Example training predictions of 10 RACER instances with true labels.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}	True
x_1	1	0	1	1	0	1	0	0	1	1	1
x_2	0	0	0	1	0	0	0	1	0	0	0
x_3	1	1	1	0	1	1	1	0	1	1	1
x_4	0	0	1	0	0	0	0	0	0	0	0
x_5	1	0	1	1	1	0	1	0	1	0	1

The predictions of the 10 RACER instances are treated as 10 input features and the true labels as the target. The logistic regression model is trained on this data and learns a weight w_i for each RACER instance; suppose it learns the weights in Table 10.

Table 10: Learned weights of the logistic-regression meta-model for each RACER instance.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
w_i	0.8	0.1	0.9	-0.2	0.3	0.7	0.2	-0.4	0.85	0.5

Note that R_4 and R_8 receive negative weights, indicating that the model has learned their predictions to be often unreliable. Now consider a test instance whose previously obtained predictions from the 10 RACER models are shown in Table 11.

Table 11: Example test-instance predictions for the logistic-regression meta-model.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
Prediction	1	0	1	0	0	1	0	1	1	0

These predictions are fed into the trained meta-model, which computes

$$\begin{aligned} S &= (0.8 \times 1) + (0.1 \times 0) + (0.9 \times 1) + (-0.2 \times 0) + (0.3 \times 0) \\ &\quad + (0.7 \times 1) + (0.2 \times 0) + (-0.4 \times 1) + (0.85 \times 1) + (0.5 \times 0) \\ &= 0.8 + 0.9 + 0.7 - 0.4 + 0.85 = 2.85. \end{aligned} \tag{9}$$

Applying the sigmoid function gives $\sigma(2.85) \approx 0.95$. Since $0.95 > 0.5$, the final prediction is class 1. The same process is applied to every test instance to determine its final label.

4.5.2. Stacking with Random Forest

This method follows the same scheme but uses a random forest[13] as the meta-model. Unlike logistic regression, which learns a fixed weight for each classifier regardless of what the others predict, a random forest can learn conditional interactions between classifiers: it learns when to trust each RACER classifier depending on what the others have predicted. The random-forest meta-model treats the predictions of the n RACER classifiers as n input features and builds an ensemble of T decision trees, each grown to a maximum depth d_{RF} on a random subset of these features and learning which combinations of their predictions lead to the correct label. At test time, the predictions of the n RACER classifiers are passed through all T trees, each tree casts a vote, and the majority vote determines the final label.

For example, using the same 10 RACER instances, suppose we have the training predictions in Table 12.

Table 12: Example training predictions of 10 RACER instances with true labels.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}	True
x_1	1	0	1	1	0	1	0	0	1	1	1
x_2	0	1	0	0	1	0	1	1	0	0	0
x_3	1	1	0	0	1	1	0	0	1	1	1
x_4	0	0	1	1	0	0	1	1	0	0	0
x_5	1	0	1	0	1	0	1	0	1	0	1
x_6	1	1	0	1	0	1	0	1	0	0	0
x_7	0	1	1	0	0	0	1	1	0	0	0
x_8	1	0	0	0	1	1	0	0	1	1	1

To illustrate how one of the T trees might work, consider a single decision tree that has selected R_1, R_9, and R_5 as its features and learned the rules in Figure 5.

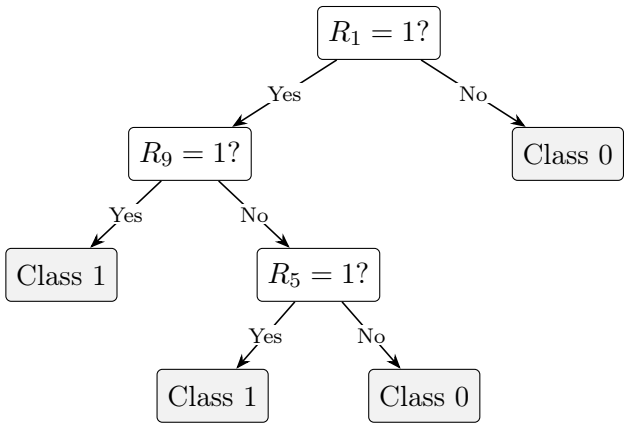


Figure 5: An example decision tree from the random-forest meta-model using features R_1, R_9, and R_5.

Tracing each training instance through this tree gives the paths in Ta-

ble 13.

Table 13: Tracing training instances through the example decision tree.

Instance	R_1	R_9	R_5	True	Tree path
x_1	1	1	–	1	R_1=1 → R_9=1 → Class 1 (correct)
x_2	0	–	–	0	R_1=0 → Class 0 (correct)
x_3	1	1	–	1	R_1=1 → R_9=1 → Class 1 (correct)
x_4	0	–	–	0	R_1=0 → Class 0 (correct)
x_5	1	1	–	1	R_1=1 → R_9=1 → Class 1 (correct)
x_6	1	0	0	0	R_1=1 → R_9=0 → R_5=0 → Class 0 (correct)
x_7	0	–	–	0	R_1=0 → Class 0 (correct)
x_8	1	1	–	1	R_1=1 → R_9=1 → Class 1 (correct)

This tree reveals a non-linear pattern: when R_1 predicts class 1, the final decision depends on the combination of R_9 and R_5. If R_9 also predicts class 1, the output is class 1; if R_9 predicts class 0, the decision falls to R_5. Such a conditional interaction between classifiers cannot be captured by logistic regression, which assigns each classifier a fixed weight regardless of the others. Now consider a test instance with $R_1 = 1$, $R_9 = 0$, and $R_5 = 1$. Following the tree, R_1=1 (Yes) → R_9=1? (No) → R_5=1? (Yes) → Class 1. This is the output of a single tree; the final prediction is decided by majority vote over all T trees, and the procedure is repeated for every test instance.

4.5.3. Stacking with Gradient Boosting

This method uses a gradient boosting [33] model as the meta-model. Unlike a random forest, which builds independent trees and combines their

results, gradient boosting builds trees sequentially, where each new tree focuses on correcting the errors of the previous ones. As before, the meta-model uses T trees, here each grown to a maximum depth d_{GB}. For example, using the same n RACER classifiers, suppose we have the training predictions in Table 14.

Table 14: Example training predictions of 10 RACER instances with true labels.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}	True
x_1	1	0	1	1	0	1	0	0	1	1	1
x_2	0	1	0	0	1	0	1	1	0	0	0
x_3	1	1	0	0	1	1	0	0	1	1	1
x_4	0	0	1	1	0	0	1	1	0	0	0
x_5	1	0	1	0	1	0	1	0	1	0	1
x_6	1	1	0	1	0	1	0	1	0	0	0
x_7	0	1	1	0	0	0	1	1	0	0	0
x_8	1	0	0	0	1	1	0	0	1	1	1

Suppose the first tree uses R_1 and R_9, producing the predictions in Table 15.

Table 15: Predictions of Tree 1 in gradient boosting using R_1 and R_9.

Instance	R_1	R_9	True	Tree 1	Correct?
x_1	1	1	1	1	Yes
x_2	0	0	0	0	Yes
x_3	1	1	1	1	Yes
x_4	0	0	0	0	Yes
x_5	1	1	1	1	Yes
x_6	1	0	0	1	No
x_7	0	0	0	0	Yes
x_8	1	1	1	1	Yes

Tree 1 misclassifies x_6 because both R_1 and R_9 are unreliable for this instance. In the next iteration, gradient boosting builds a second tree that focuses on this error, using R_5 and R_{10} to correct it, as shown in Table 16.

Table 16: Sequential error correction by Trees 1 and 2 in gradient boosting.

Instance	R_1	R_9	R_5	R_{10}	True	Tree 1	Tree 2	Final
x_1	1	1	0	1	1	1	1	1 (correct)
x_2	0	0	1	0	0	0	0	0 (correct)
x_3	1	1	1	1	1	1	1	1 (correct)
x_4	0	0	0	0	0	0	0	0 (correct)
x_5	1	1	1	0	1	1	1	1 (correct)
x_6	1	0	0	0	0	1	0	0 (correct)
x_7	0	0	0	0	0	0	0	0 (correct)
x_8	1	1	1	1	1	1	1	1 (correct)

By building trees sequentially, gradient boosting gradually corrects the mistakes of earlier trees. Here, Tree 1 relied on R_1 and R_9 but failed on x_6, and Tree 2 then used R_5 and R_{10} to fix that error. This step-by-step correction lets the model handle difficult instances that a single tree might miss. During testing, the predictions of the 10 RACER models are fed into the trained gradient boosting model, which combines the outputs of all its sequential trees to determine the final label of each test instance.

4.6. The MORCE Algorithm

Having described the base classifiers, the prediction matrix, and the aggregation strategies, we now summarize the complete MORCE procedure. Algorithm 1 outlines the full process, from generating the random orderings and training the individual RACER classifiers to aggregating their predictions into the final labels.

Algorithm 1 MORCE (Multi-Ordering Rule-based Classifier Ensemble)

Require: Training set D_{train}, test set D_{test}, number of RACER classifiers n, aggregation method $\mathcal{A}$

Ensure: Predicted labels for the test set

1: **for** $i = 1$ to n **do**
2: Shuffle D_{train} into a random ordering for classifier C_i
3: Train C_i on this ordering to obtain its rule set
4: Record the training accuracy of C_i
5: Use the rule set of C_i to predict the labels of D_{test}
6: **end for**
7: Combine the predictions of $C_1, \ldots, C_n$ using $\mathcal{A}$
8: **return** the resulting labels

5. Experiments and Results

In this section, we present the experimental evaluation of the proposed MORCE method. We describe the experimental setup, the datasets used, and analyze the results obtained with the different aggregation methods. Performance is assessed using two complementary metrics: classification accuracy and the F_1 score. Accuracy is defined as the fraction of correctly classified test instances:

$$\text{Accuracy} = \frac{\text{Number of correctly classified test instances}}{\text{Total number of test instances}} \tag{10}$$

Because several of the datasets used in this study exhibit class imbalance, accuracy alone can overstate the quality of a classifier that favours the majority class. We therefore also report the F_1 score, which combines precision and recall into a single value through their harmonic mean:

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{11}$$

where precision measures the proportion of predicted positive instances that are actually correct, and recall measures the proportion of actual positive instances that the classifier successfully identifies:

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN} \tag{12}$$

Here TP, FP, and FN denote the numbers of true positives, false positives, and false negatives, respectively. The harmonic mean is used rather than the arithmetic mean because it penalizes low values more heavily, so a high F_1 score requires both precision and recall to be high. For datasets with more than two classes, the F_1 score is computed for each class separately and then

combined as a weighted average, where each class contributes in proportion to its number of instances in the test set:

$$F_1^{weighted} = \sum_{c \in C} \frac{n_c}{N} \times F_{1,c} \tag{13}$$

where $F_{1,c}$ is the F_1 score of class c, n_c is the number of test instances belonging to class c, $|C|$ is the number of classes, and $N = \sum_{c \in C} n_c$ is the total number of test instances. Reporting both accuracy and F_1 therefore gives a more complete picture of ensemble behaviour than accuracy alone.

5.1. Experimental Setup

The evaluation of the proposed ensemble approach involves several key design choices, each of which is described below.

Number of Orderings. The number of random orderings, and hence the number of base RACER classifiers, is set to $n = 30$ in all experiments.

Cross-Validation. To ensure a reliable estimation of classification performance, we employ 10-fold cross-validation [34]. The dataset is divided into 10 equal-sized partitions. In each iteration, 9 partitions are used for training and the remaining one is used for testing. This process is repeated 10 times, with each partition serving as the test set exactly once. In each fold, the ensemble method produces a final predicted label for each test instance, as described in the previous section, and both accuracy and the F_1 score of the ensemble are computed for that fold. The final value of each metric is then obtained by averaging over all 10 folds.

Stacking Meta-model Parameters. The random forest and gradient boosting meta-models each use $T = 50$ trees, with a maximum depth of $d_{RF} = 5$ for the random forest and $d_{GB} = 3$ for gradient boosting; logistic regression uses the default regularization.

RACER Parameters. Each RACER classifier is configured with $\alpha = 0.99$. The high value of α prioritizes rule accuracy over coverage, encouraging the generation of precise classification rules.

Top-k Selection. For the top-k voting and top-k weighted voting methods, k is set to 5 in all experiments, so that only the five most accurate of the $n = 30$ classifiers participate in the final decision.

5.2. Datasets

The experiments are conducted on 23 datasets sourced from the UCI Machine Learning Repository [35] and OpenML [36]. These datasets vary in the number of instances (from 90 to 1473), number of features (from 7 to 1204), and number of classes (from 2 to 6). Table 17 summarizes the characteristics of the datasets used in this study.

Table 17: Summary of datasets used in the experiments.

No.	Dataset	#Inst.	#Feat.	#Classes	#Mis. val.	#Num. attr.	#Cat. attr.
1	Period-Changer	90	1178	2	0	1177	0
2	Somerville Happiness	143	7	2	0	6	0
3	lymph	148	19	4	0	3	15
4	Toxicity	171	1204	2	0	1203	0
5	DARWIN	174	452	2	0	450	1
6	parkinsons	195	23	2	0	22	0
7	pwLinear	200	11	2	0	10	0
8	heart-statlog	270	14	2	0	13	0
9	breastTumor	286	10	2	9	3	6
10	Neurofibromatosis	296	19	2	73	18	0
11	Z-Alizadeh sani	303	56	2	0	34	21
12	cholesterol	303	14	2	6	13	0
13	cleveland	303	14	2	6	13	0
14	dermatology	366	35	6	8	34	0
15	pbc	418	19	2	1239	18	0
16	vote	438	17	2	440	0	16
17	kc2	522	22	2	0	21	0
18	eucalyptus	736	20	2	448	14	5
19	diabetes	768	9	2	0	8	0
20	Titanic	891	8	2	0	7	0
21	tokyo1	959	45	2	0	44	0
22	SWD	1000	11	4	0	10	0
23	cmc	1473	10	2	0	9	0

5.3. Results

Table 18 presents the classification accuracy of the baseline RACER algorithm and the seven proposed aggregation methods across all 23 datasets. The best value in each row is shown in bold. Among the seven methods, Stacking (GB) achieves the best overall performance with an average accu-

racy of 77.53%, followed closely by Stacking (RF) with an average accuracy of 77.50%, both improving over the baseline of 75.84%.

Table 18: Classification accuracy (%) of the baseline RACER and the proposed MORCE aggregation methods.

			MORCE						
No.	Dataset	RACER	Voting	Top-k Voting	Weighted Voting	Top-k Weighted Voting	Stacking (LR)	Stacking (RF)	Stacking (GB)
1	lymph	77.71	81.71	81.09	81.71	81.09	82.38	81.04	**83.04**
2	pwLinear	88.00	**88.50**	**88.50**	**88.50**	**88.50**	**88.50**	**88.50**	**88.50**
3	heart-statlog	78.88	**80.00**	79.25	**80.00**	79.25	79.25	**80.00**	**80.00**
4	breastTumor	55.87	56.21	**56.24**	56.21	**56.24**	56.23	55.88	**56.24**
5	cholesterol	60.37	60.37	**61.04**	60.37	**61.04**	60.70	60.70	60.70
6	cleveland	75.88	76.86	76.55	76.86	76.55	76.88	**78.16**	76.53
7	pbc	67.46	69.59	68.64	**69.83**	68.64	69.36	69.59	69.12
8	kc2	83.91	84.29	84.10	84.29	84.10	84.29	**84.48**	84.29
9	cmc	**66.87**	66.19	66.60	66.32	66.60	66.73	66.60	66.60
10	vote	**95.89**	**95.89**	95.66	95.44	95.66	95.21	95.21	95.44
11	diabetes	71.75	70.84	**72.92**	70.84	**72.92**	72.66	72.66	72.79
12	tokyo1	91.55	91.97	92.49	91.97	92.49	92.07	92.17	**92.59**
13	dermatology	94.77	95.05	**95.33**	95.05	**95.33**	94.24	95.05	**95.33**
14	parkinsons	86.68	87.73	**88.26**	87.73	**88.26**	87.71	86.71	88.23
15	Somerville Happiness	60.28	**60.95**	**60.95**	**60.95**	**60.95**	**60.95**	**60.95**	**60.95**
16	Neurofibromatosis	54.40	55.73	**56.09**	55.73	**56.09**	**56.09**	56.06	55.73
17	SWD	51.09	50.70	54.80	50.80	55.09	54.80	56.39	**57.09**
18	Titanic	75.97	75.86	78.33	75.86	78.33	**78.55**	78.33	**78.55**
19	Period-Changer	80.00	**81.11**	**81.11**	**81.11**	**81.11**	80.00	**81.11**	**81.11**
20	Toxicity	81.89	86.50	83.00	86.50	83.00	85.32	**86.53**	84.77
21	DARWIN	87.97	91.99	**93.69**	91.99	**93.69**	91.99	91.40	90.84
22	Z-Alizadeh sani	74.58	79.54	79.51	79.54	79.51	78.90	**80.21**	80.20
23	Thyroid_Diff	82.50	84.33	82.51	84.33	82.51	84.34	**84.85**	84.59
	Average	75.84	77.04	77.24	77.04	77.26	77.27	77.50	**77.53**

Table 19 reports the corresponding F_1 scores for the same methods and datasets. The ranking of the aggregation strategies is consistent with that observed for accuracy: Stacking (GB) again attains the highest average F_1 score at 76.55%, followed by Stacking (RF) at 76.50%, while the baseline RACER reaches 74.77%.

Table 19: F_1 score (%) of the baseline RACER and the proposed MORCE aggregation methods.

			MORCE						
No.	Dataset	RACER	Voting	Top-k Voting	Weighted Voting	Top-k Weighted Voting	Stacking (LR)	Stacking (RF)	Stacking (GB)
1	lymph	76.15	80.11	79.61	80.11	79.61	80.69	79.45	**81.35**
2	pwLinear	87.94	**88.43**	**88.43**	**88.43**	**88.43**	**88.43**	**88.43**	**88.43**
3	heart-statlog	79.27	80.36	79.61	80.36	79.61	79.65	80.36	**80.37**
4	breastTumor	55.58	56.00	56.20	56.00	56.20	**56.23**	55.51	56.07
5	cholesterol	60.21	60.10	**60.70**	60.10	**60.70**	60.52	60.52	60.52
6	cleveland	75.90	76.85	76.58	76.85	76.58	76.90	**78.12**	76.56
7	pbc	67.26	69.34	68.14	**69.57**	68.14	68.90	68.99	68.73
8	kc2	83.15	83.64	83.38	83.64	83.38	83.55	**83.68**	83.46
9	cmc	63.55	62.77	63.50	62.98	63.50	**63.87**	63.65	63.65
10	vote	95.91	**95.92**	95.68	95.48	95.68	95.25	95.24	95.47
11	diabetes	68.13	66.94	70.21	67.53	70.21	70.19	70.19	**70.31**
12	tokyo1	91.52	91.95	92.49	91.95	92.49	92.07	92.17	**92.58**
13	dermatology	94.47	94.76	95.03	94.76	95.03	93.89	94.76	**95.06**
14	parkinsons	86.21	86.71	87.34	86.71	87.34	86.92	85.73	**87.43**
15	Somerville Happiness	60.05	**60.67**	**60.67**	**60.67**	**60.67**	**60.67**	**60.67**	**60.67**
16	Neurofibromatosis	54.44	55.49	**56.06**	55.49	**56.06**	56.02	56.01	55.67
17	SWD	48.35	47.71	53.22	48.20	53.56	53.99	55.58	**56.53**
18	Titanic	76.19	76.11	78.22	76.11	78.22	**78.43**	78.13	78.35
19	Period-Changer	74.82	**76.45**	**76.45**	**76.45**	**76.45**	74.82	**76.45**	**76.45**
20	Toxicity	80.55	**84.93**	80.91	**84.93**	80.91	83.58	84.91	82.82
21	DARWIN	87.96	92.00	**93.71**	92.00	**93.71**	92.00	91.44	90.86
22	Z-Alizadeh sani	70.65	75.22	75.67	75.22	75.67	74.08	76.00	**76.15**
23	Thyroid_Diff	81.49	83.02	81.50	83.02	81.50	83.22	**83.62**	83.22
	Average	74.77	75.89	76.23	75.94	76.25	76.25	76.50	**76.55**

Having compared the seven aggregation strategies against one another, we now position the best of them against established classifiers. In the remainder of this section, MORCE refers to stacking with gradient boosting, the variant that achieved the highest average on both metrics. It is compared against ten well-known classifiers spanning the main families of supervised learning: the baseline RACER together with its two recent extensions ROPAC-M and ROPAC-L, the ensemble methods Random Forest and Bagging, and the single-model learners MLP, SVM, CART, IBk, and Naïve Bayes. All classifiers were evaluated under identical conditions, using the same 10-fold cross-validation splits on the same 23 datasets.

Table 20 presents the classification accuracy of MORCE compared against these ten classifiers across all 23 datasets.

Table 20: Accuracy comparison of classifiers (%).

Dataset	RACER	ROPAC-M	ROPAC-L	**MORCE**	Random Forest	MLP	Bagging	SVM	CART	IBk	Naïve Bayes
lymph	77.71	77.71	77.71	**83.04**	**83.04**	78.23	80.28	79.61	75.52	79.66	71.47
pwLinear	88.00	84.00	86.99	**88.50**	**88.50**	**88.50**	87.50	85.50	82.00	79.00	84.00
heart-statlog	78.88	79.25	78.51	80.00	81.48	77.03	80.37	81.85	78.51	82.96	**83.33**
breastTumor	55.87	53.47	57.96	56.24	56.63	54.63	56.25	59.11	52.80	53.11	**62.59**
cholesterol	60.37	60.03	60.08	**60.70**	56.09	56.09	55.83	59.04	55.43	57.40	60.03
cleveland	75.88	75.53	75.21	76.53	81.82	78.52	81.20	82.79	74.92	82.80	**84.13**
pbc	67.46	67.46	66.97	69.12	69.37	68.36	**70.80**	70.32	64.06	65.76	66.44
kc2	83.91	83.91	84.10	**84.29**	83.36	82.39	83.17	83.73	78.74	82.39	83.53
cmc	66.87	66.32	66.46	66.60	67.47	69.58	68.42	**71.08**	63.27	65.71	63.34
vote	95.89	**96.12**	94.98	95.44	95.43	94.74	95.21	95.67	93.83	93.59	91.55
diabetes	71.75	70.06	70.57	72.79	**76.71**	73.43	75.01	75.66	72.54	74.22	75.79
tokyo1	91.55	91.86	92.59	92.59	**93.42**	91.44	92.38	92.28	89.25	92.39	88.21
dermatology	94.77	95.06	94.77	95.33	96.98	96.71	96.44	**97.25**	93.17	95.62	87.71
parkinsons	86.68	86.21	86.21	88.23	91.84	**93.84**	91.84	87.76	87.76	90.31	71.21
Somerville Happiness	60.28	58.14	60.28	60.95	**62.42**	55.28	61.66	59.57	58.09	51.28	58.95
Neurofibromatosis	54.40	53.39	54.74	**55.73**	47.04	53.39	45.71	47.34	50.72	48.33	55.42
SWD	51.09	49.70	49.40	57.09	**57.49**	57.40	57.40	57.00	57.09	53.49	56.50
Titanic	75.97	74.18	78.78	78.55	82.37	81.14	82.15	**83.27**	81.25	82.37	79.79
Period-Changer	80.00	80.00	80.00	**81.11**	63.33	58.88	64.44	71.11	47.77	64.44	41.11
Toxicity	81.89	81.89	81.89	**84.77**	63.07	52.02	60.16	66.04	59.01	61.92	40.98
DARWIN	87.97	87.97	87.97	**90.84**	86.86	85.19	84.60	86.20	77.51	70.16	86.79
Z-Alizadeh sani	74.58	74.58	74.58	80.20	**87.12**	85.12	84.84	85.47	80.16	80.87	74.55
Thyroid_Diff	82.50	83.02	83.02	84.59	**96.07**	93.19	95.79	94.51	93.72	89.81	89.28
AVG	75.84	75.21	75.82	**77.53**	76.87	75.00	76.15	77.05	72.48	73.81	72.03

The same comparison is now repeated for the F_1 score. Table 21 reports

the results across the same 23 datasets and ten classifiers.

Table 21: F_1 score comparison of classifiers (%).

Dataset	RACER	ROPAC-M	ROPAC-L	**MORCE**	Random Forest	MLP	Bagging	SVM	CART	IBk	Naïve Bayes
lymph	76.15	76.15	76.15	81.35	**81.99**	76.98	79.08	77.69	74.57	78.47	68.81
pwLinear	87.94	83.83	86.94	88.43	88.50	**88.51**	87.50	85.43	81.96	78.80	83.99
heart-statlog	79.27	79.65	78.96	80.37	81.64	77.28	80.62	81.98	78.83	83.09	**83.52**
breastTumor	55.58	53.04	57.93	56.07	55.95	54.46	55.33	58.38	52.85	52.52	**62.89**
cholesterol	60.21	59.55	59.73	**60.52**	55.64	55.19	55.46	58.23	54.81	57.12	59.19
cleveland	75.90	75.55	75.22	76.56	81.79	78.55	81.19	82.73	74.97	82.77	**84.05**
pbc	67.26	67.35	66.59	68.73	69.43	68.41	**70.87**	70.42	64.22	65.74	65.68
kc2	83.15	83.15	83.32	**83.46**	82.14	81.37	82.09	81.57	78.02	81.82	81.68
cmc	63.55	63.13	62.41	63.65	67.11	69.06	68.15	**70.07**	63.22	64.66	61.45
vote	95.91	**96.13**	95.01	95.47	95.44	94.74	95.20	95.69	93.85	93.62	91.60
diabetes	68.13	66.79	69.19	70.31	**76.23**	73.24	74.74	74.87	72.46	73.59	75.40
tokyo1	91.52	91.83	92.61	92.58	**93.40**	91.41	92.35	92.22	89.23	92.38	87.91
dermatology	94.47	94.79	94.47	95.06	96.94	96.73	96.45	**97.27**	93.24	95.66	86.08
parkinsons	86.21	85.41	85.35	87.43	91.33	**93.87**	91.47	85.79	87.97	90.26	73.13
Somerville Happiness	60.05	57.85	59.93	60.67	**61.81**	54.09	60.85	58.41	57.46	50.65	58.52
Neurofibromatosis	54.44	53.43	54.83	**55.67**	46.91	53.02	45.29	42.20	50.67	47.82	52.94
SWD	48.35	46.78	45.06	56.53	**57.19**	57.12	57.11	56.00	56.77	52.93	56.29
Titanic	76.19	74.46	78.49	78.35	82.10	80.87	81.91	**82.85**	80.81	82.11	79.94
Period-Changer	74.82	74.82	74.82	**76.45**	59.30	59.87	60.52	62.77	49.26	62.67	40.14
Toxicity	80.55	80.55	80.55	**82.82**	54.90	50.26	53.69	53.70	58.48	60.00	36.32
DARWIN	87.96	87.96	87.96	**90.86**	86.97	84.94	84.48	86.26	77.49	67.50	86.81
Z-Alizadeh sani	70.65	70.65	70.65	76.15	**86.59**	85.00	84.83	84.99	80.57	80.77	75.74
Thyroid_Diff	81.49	82.05	82.12	83.22	**96.00**	93.17	95.79	94.39	93.76	89.39	89.05
AVG	74.77	74.13	74.71	**76.55**	76.06	74.70	75.43	75.39	72.41	73.23	71.35

According to Tables 20 and 21, MORCE attains the highest average performance on both metrics, reaching 77.53% accuracy and 76.55% F_1, and

thereby outperforming all ten competing classifiers. It raises the base learner from 75.84% to 77.53% in accuracy and from 74.77% to 76.55% in F_1, lifting RACER from fifth to first place in the overall ranking of the eleven classifiers. On accuracy, the closest competitors are SVM (77.05%) and Random Forest (76.87%), which MORCE exceeds by 0.48 and 0.66 percentage points respectively; on F_1, Random Forest (76.06%) and Bagging (75.43%) follow. MORCE also improves on both RACER extensions, surpassing ROPAC-L by 1.71 and ROPAC-M by 2.32 accuracy points.

The per-dataset results reveal where the improvement originates. MORCE achieves the best accuracy of all eleven classifiers on eight datasets, and its largest margins occur on the high-dimensional, small-sample problems: on Toxicity it reaches 84.77% against 63.07% for Random Forest and 52.02% for MLP, and on Period-Changer 81.11% against 63.33% and 58.88% respectively. These are precisely the settings in which the statistical learners struggle, whereas the rule-based ensemble remains robust. Conversely, on datasets such as Thyroid_Diff and Z-Alizadeh sani the tree- and margin-based methods retain a clear advantage, indicating that MORCE complements rather than uniformly dominates them.

5.4. Performance Improvement Analysis

To better understand the effectiveness of each aggregation method, we compare its average performance with that of the baseline RACER. For each metric, the improvement is computed as:

$$\Delta_M = \text{Avg}_M(\text{Aggregation Method}) - \text{Avg}_M(\text{RACER}), \qquad M \in \{\text{Accuracy}, F_1\} \tag{14}$$

A positive value indicates that the aggregation method outperforms the baseline on that metric, while a negative value indicates degradation. Both Δ_{Accuracy} and Δ_{F_1} are reported, so that improvements achieved at the expense of minority-class performance can be distinguished from genuine gains. Figure 6 summarizes these results, showing the average accuracy and F_1 score of each aggregation method against the RACER baseline (dotted line) across all 23 datasets.

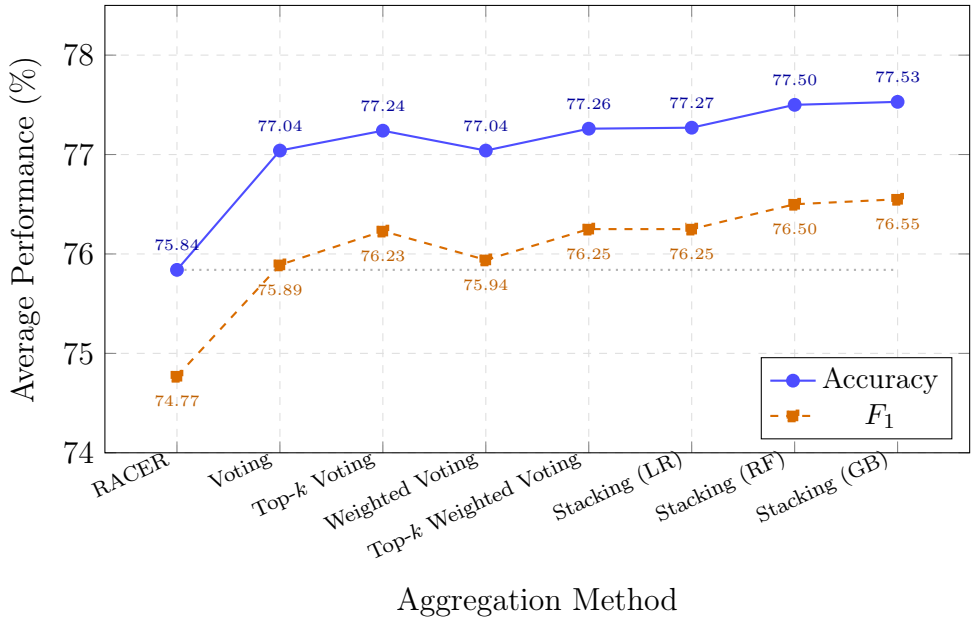
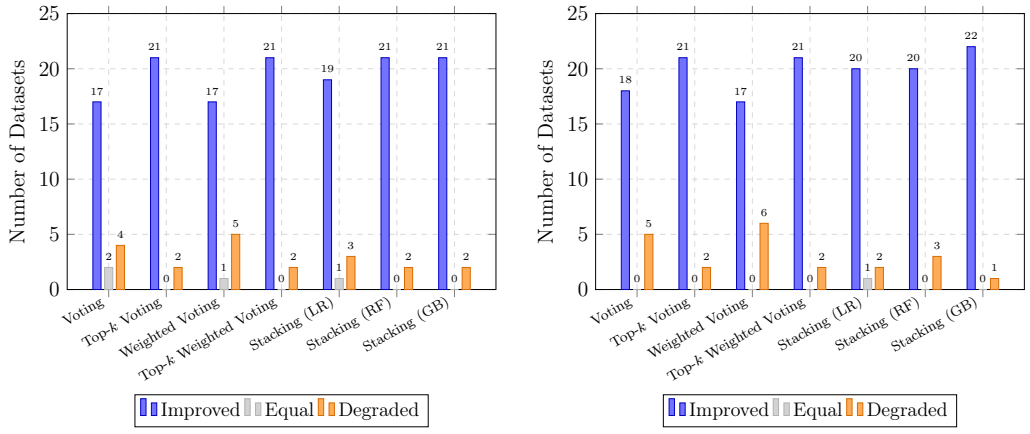


Figure 6: Average accuracy and F_1 score (%) of each aggregation method compared to the baseline RACER across 23 datasets.

Figure 7 shows, for each aggregation method, the number of datasets on which the baseline RACER is improved, matched, or degraded, under accuracy (Figure 7a) and F_1 (Figure 7b). The two top-k variants and the tree-based stacking models improve the baseline on 21 of the 23 datasets under

accuracy, and Stacking (GB) on 22 under F_1, while plain majority voting and weighted voting improve on only 17 each — confirming that selecting the strongest orderings is more effective than weighting.



(a) Accuracy

(b) F_1 score

Figure 7: Number of datasets (out of 23) where each aggregation method improves, matches, or degrades the performance of the baseline RACER.

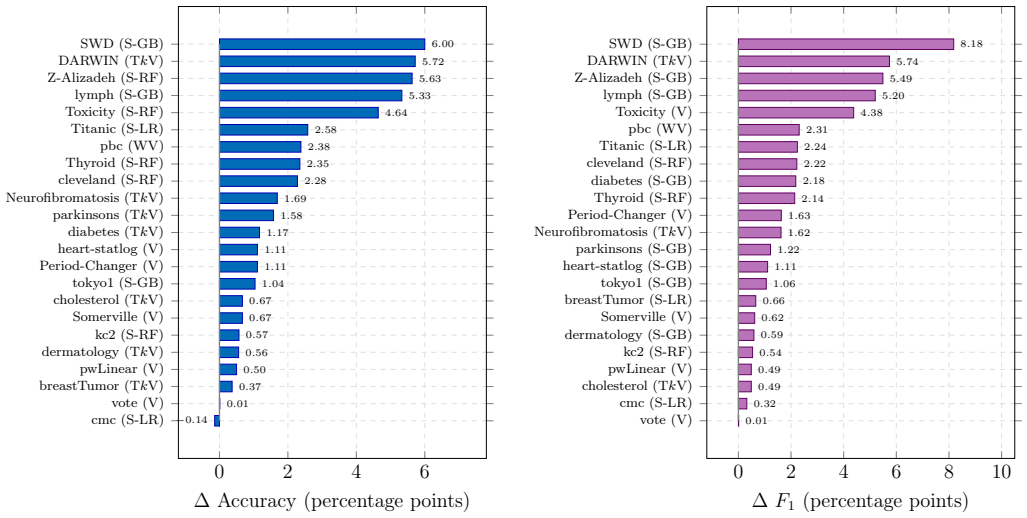
Figure 8 shows the per-dataset gain of the best-performing MORCE aggregation method over the baseline RACER, sorted by magnitude, for accuracy (Figure 8a) and F_1 (Figure 8b); the method achieving the best result on each dataset is indicated next to the dataset name. Selecting the aggregation method per dataset raises the average gain to 2.08 percentage points in accuracy and 2.19 in F_1, compared to 1.69 and 1.78 when a single method (Stacking GB) is applied uniformly. The gains are concentrated where RACER is weakest — SWD, Z-Alizadeh sani, DARWIN, and lymph each improve by more than five percentage points on both metrics — while the only dataset that falls below the baseline (cmc, and only in accuracy) loses less than 0.15 points, and even recovers a positive F_1 gain. This indicates that the ensemble

is most valuable precisely where the ordering sensitivity of a single RACER model is most damaging, and that the risk of aggregation is bounded: no dataset is meaningfully harmed.

No single aggregation strategy dominates. Stacking wins most often under F_1 — gradient boosting on eight datasets and logistic regression and random forest on three each — whereas under accuracy the simpler Top-k Voting is best on six. Plain voting remains the best choice on several datasets, either because the base rule sets already agree and a learned meta-classifier only adds variance (vote, where every method lies within 0.5 points of the baseline), or because a plain majority over many diverse rule sets is enough to resolve their disagreement without the extra parameters stacking introduces (Toxicity, +4.38 points in F_1). Ties are broken in favour of the simpler method; in particular, Top-k Voting and Top-k Weighted Voting produce identical predictions on 22 of the 23 datasets, and weighted voting differs from plain voting on only four. This suggests that the confidence weights are nearly uniform among the highest-quality rules, so the benefit of MORCE stems from combining multiple complementary rule sets rather than from how their votes are weighted.

The absence of a universally best aggregation method is therefore not a limitation of MORCE but a property of the framework itself: the aggregation step is a free design choice that can be tuned to the dataset at hand, and the diversity of winning strategies indicates that the base rule sets carry complementary information which different aggregators exploit in different ways. MORCE is thus better viewed as a family of classifiers sharing a common pool of base rule sets, with the aggregation strategy acting as an additional

hyperparameter that can be selected by cross-validation on the training folds at negligible cost, since the rule sets are learned once and reused by every aggregator. The per-dataset best results reported here are intended as an upper bound on what such a selection procedure could achieve, while the uniform per-method averages in Tables 18 and 19 provide the corresponding lower bound.



(a) Accuracy

(b) F_1 score

Figure 8: Per-dataset improvement of the best-performing MORCE aggregation method over the baseline RACER, sorted by magnitude. The method achieving the best result on each dataset is shown in parentheses and is selected independently for each metric (V: Voting, TkV: Top-k Voting, TkWV: Top-k Weighted Voting, WV: Weighted Voting, S-LR/S-RF/S-GB: Stacking with logistic regression, random forest, and gradient boosting).

All seven aggregation methods improve the average performance over the baseline on both metrics. The stacking-based strategies prove the most effective: Stacking (GB) attains the highest averages at 77.53% accuracy and 76.55% F_1, followed by Stacking (RF) at 77.50% and 76.50%. Among the

voting-based strategies, the benefit of top-k selection is clearly visible — accuracy rises from 77.04% to 77.24% when only the k most accurate classifiers vote, and the number of datasets on which the baseline is improved rises from 18 to 21 (from 17 to 21 when the votes are also weighted). Weighting the votes by training accuracy, by contrast, leaves the average essentially unchanged (77.04% in both cases), and the same holds once top-k selection is applied. This indicates that the useful signal in the training accuracies lies in *which* classifiers to retain rather than in how strongly to weight them. That the second-level models outperform every voting scheme suggests that learning conditional combination patterns over all n orderings extracts more information than discarding the weaker ones outright.

To verify that the observed improvements are statistically significant, a Wilcoxon signed-rank test [37] is performed between each aggregation method and the baseline RACER across the 23 datasets. As shown in Table 22, all seven methods improve over the baseline at the 0.01 significance level on both metrics. The two top-k voting schemes and the two tree-based stacking models yield the strongest evidence, with p-values below 10^{-4} on at least one metric, while plain and weighted majority voting — the two methods that treat all thirty orderings alike — return the weakest.

Table 22: Comparison of each aggregation method against the baseline RACER over the 23 datasets (Wilcoxon signed-rank test).

	Accuracy				F_1			
Method	Avg. (%)	Δ	W / T / L	p-value	Avg. (%)	Δ	W / T / L	p-value
Voting	77.04	+1.20	17 / 2 / 4	0.00155	75.89	+1.12	18 / 0 / 5	0.00307
Top-k Voting	77.24	+1.40	21 / 0 / 2	< 0.0001	76.23	+1.46	21 / 0 / 2	< 0.0001
Weighted Voting	77.04	+1.20	17 / 1 / 5	0.00193	75.94	+1.17	17 / 0 / 6	0.00098
Top-k Weighted Voting	77.26	+1.42	21 / 0 / 2	< 0.0001	76.25	+1.48	21 / 0 / 2	< 0.0001
Stacking (LR)	77.27	+1.43	19 / 1 / 3	0.00048	76.25	+1.48	20 / 1 / 2	0.00033
Stacking (RF)	77.50	+1.66	21 / 0 / 2	0.00014	76.50	+1.73	20 / 0 / 3	0.00004
Stacking (GB)	77.53	+1.69	21 / 0 / 2	0.00006	76.55	+1.78	22 / 0 / 1	< 0.0001

6. Conclusion

In this paper, MORCE was proposed as an ensemble-based extension of the RACER algorithm that exploits its sensitivity to training data ordering. Presenting the same training set in a different order yields a different rule set, and therefore different predictions on the same test instances. What is conventionally regarded as a weakness — that a single RACER model is not reproducible across permutations of its input — was here turned into a source of ensemble diversity: multiple independent RACER classifiers were trained on random orderings of the same data and their predictions combined through voting-based and stacking-based strategies. All seven aggregation methods improved the average accuracy and F_1 score over the baseline, every improvement significant at the 0.01 level under the Wilcoxon signed-rank test (Table 22). Stacking with gradient boosting performed best on average, surpassing the baseline by 1.69 percentage points in accuracy and 1.78 in F_1, and improving on it in 21 and 22 of the 23 datasets respectively.

Three findings emerge. First, the training accuracies of the base classifiers indicate *which* orderings to retain but not how strongly to weight them: weighting altered the outcome of majority voting on only four datasets, whereas restricting the vote to the five strongest classifiers raised the number of datasets improved over the baseline from 18 to 21 in accuracy, and from 17 to 21 when the votes were also weighted. Second, a second-level model over all orderings proved more effective still than discarding the weaker ones, indicating that even low-accuracy rule sets contribute information a meta-model can exploit conditionally. Third, although Stacking (GB) is the strongest method on average, no single strategy dominates at the dataset level: selecting the aggregation method per dataset raises the average gain to 2.08 percentage points in accuracy and 2.19 in F_1, which suggests that the aggregation step is best treated as a hyperparameter of the framework rather than a fixed design decision.

The per-dataset results support this interpretation. The largest gains occurred where a single ordering generalises least reliably — SWD, DARWIN, Z-Alizadeh sani, and lymph improved by 6.00, 5.72, 5.63, and 5.33 percentage points — while on datasets such as vote and pwLinear, where RACER is already near its ceiling, the orderings agree almost everywhere and aggregation has little diversity to exploit; on pwLinear all seven methods return an identical result. Against ten well-known classifiers, MORCE lifted RACER from fifth to first place in average accuracy and F_1 while leaving the base classifier untouched, with its advantage most pronounced on high-dimensional, small-sample datasets — Toxicity, Period-Changer, and again DARWIN — where the statistical learners degraded sharply and the rule-based ensemble

remained robust.

Despite these results, the approach has limitations, and several avenues remain open for future investigation. The principal limitation is runtime: training $n = 30$ independent RACER models increases the cost of rule induction by roughly a factor of n, and the overhead grows quickly with dataset size. Devising a more efficient training scheme — for instance by exploiting the mutual independence of the orderings, or by reducing the number of models that must be trained — would therefore be a valuable extension of this work.

Beyond efficiency, several directions merit investigation. Both n and k were held constant in this study, yet the useful number of orderings evidently depends on how unstable the base learner is on a given dataset, so determining k adaptively from the spread of training accuracies — or halting once the ensemble prediction stabilises — would improve both accuracy and cost. The same argument applies to the choice of aggregation method, which could be selected by cross-validation on the training folds at negligible additional cost, since the rule sets are learned once and reused by every aggregator.The diversity induced by ordering could also be characterised directly through pairwise measures such as the Q-statistic [38] or kappa-error diagrams [39], and comparing it against the diversity produced by bootstrap resampling, as done for bagging and boosting in [40], would show whether the two mechanisms are complementary. Finally, the framework should extend to other ordering-sensitive rule learners: GRACER [27], ROPAC [28], and RACE-RANK [30] share the same order-dependent merging mechanism, and sequential covering methods exhibit comparable sensitivity to instance order. Establishing

whether the gains transfer would show whether multi-ordering aggregation is specific to RACER or a general principle for rule-based ensembles.

References

[1] J. R. Quinlan, Induction of decision trees, Machine Learning 1 (1) (1986) 81–106.

[2] T. M. Cover, P. E. Hart, Nearest neighbor pattern classification, IEEE Transactions on Information Theory 13 (1) (1967) 21–27.

[3] G. H. John, P. Langley, Estimating continuous distributions in Bayesian classifiers, in: Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence (UAI), 1995, pp. 338–345.

[4] C. Cortes, V. Vapnik, Support-vector networks, Machine Learning 20 (3) (1995) 273–297.

[5] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, Nature 323 (6088) (1986) 533–536.

[6] C. M. Bishop, Pattern Recognition and Machine Learning, Springer, New York, 2006.

[7] J. Basiri, F. Taghiyareh, H. Faili, RACER: Accurate and efficient classification based on rule aggregation approach, Neural Computing and Applications 31 (3) (2019) 895–908. doi:10.1007/s00521-017-3117-2.

[8] T. G. Dietterich, Ensemble methods in machine learning, in: Multiple Classifier Systems (MCS), Vol. 1857 of Lecture Notes in Computer Science, Springer, 2000, pp. 1–15.

[9] L. I. Kuncheva, Combining Pattern Classifiers: Methods and Algorithms, 2nd Edition, Wiley, Hoboken, NJ, 2014.

[10] R. Polikar, Ensemble based systems in decision making, IEEE Circuits and Systems Magazine 6 (3) (2006) 21–45.

[11] L. Breiman, Bagging predictors, Machine Learning 24 (2) (1996) 123–140.

[12] Y. Freund, R. E. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting, Journal of Computer and System Sciences 55 (1) (1997) 119–139.

[13] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32.

[14] J. R. Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufmann, San Mateo, CA, 1993.

[15] L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone, Classification and Regression Trees, Routledge, 2017, originally published 1984 by Wadsworth.

[16] M. N. Adnan, M. Z. Islam, Forest PA: Constructing a decision forest by penalizing attributes used in previous trees, Expert Systems with Applications 89 (2017) 389–403.

[17] M. N. Adnan, M. Z. Islam, Optimizing the number of trees in a decision forest to discover a subforest with high ensemble accuracy using a genetic algorithm, Knowledge-Based Systems 110 (2016) 86–97.

[18] N. Landwehr, M. Hall, E. Frank, Logistic model trees, Machine Learning 59 (1–2) (2005) 161–205.

[19] D. Yates, M. Z. Islam, J. Gao, SPAARC: A fast decision tree algorithm, in: Data Mining (AusDM 2018), Vol. 996 of Communications in Computer and Information Science, Springer, Singapore, 2019, pp. 43–55.

[20] J. Fürnkranz, Separate-and-conquer rule learning, Artificial Intelligence Review 13 (1) (1999) 3–54.

[21] T. M. Mitchell, Machine Learning, McGraw-Hill, New York, 1997.

[22] P. Clark, T. Niblett, The CN2 induction algorithm, Machine Learning 3 (4) (1989) 261–283.

[23] E. Frank, I. H. Witten, Generating accurate rule sets without global optimization, in: Proceedings of the Fifteenth International Conference on Machine Learning (ICML), Morgan Kaufmann, 1998, pp. 144–151.

[24] W. W. Cohen, Fast effective rule induction, in: Proceedings of the Twelfth International Conference on Machine Learning (ICML), Morgan Kaufmann, 1995, pp. 115–123.

[25] M. Khosravi, A. Basiri, Mining interpretable rules with MCRM: A novel rule mining algorithm with inherent feature selection and discretization, Information Sciences 698 (2025) 121785. doi:10.1016/j.ins.2024.121785.

[26] D. Karaboga, B. Basturk, A powerful and efficient algorithm for nu-

merical function optimization: Artificial bee colony (ABC) algorithm, Journal of Global Optimization 39 (3) (2007) 459–471.

[27] P. Hosseini, A. Basiri, GRACER: Improving the accuracy of RACER classifier using a greedy approach, in: 2022 27th International Computer Conference, Computer Society of Iran (CSICC), IEEE, 2022, pp. 1–6. doi:10.1109/CSICC55295.2022.9780528.

[28] M. Mokhtari, A. Basiri, ROPAC: Rule OPtimized aggregation classifier, Expert Systems with Applications 250 (2024) 123897. doi:10.1016/j.eswa.2024.123897.

[29] M. Ebrahimi, A. Basiri, RACEkNN: A hybrid approach for improving the effectiveness of the k-nearest neighbor algorithm, Knowledge-Based Systems 301 (2024) 112357. doi:10.1016/j.knosys.2024.112357.

[30] A. Basiri, N. Pournajar, RACE-RANK: A hybrid rule-based classifier using PageRank algorithm, Applied Soft Computing 186 (2025) 114213. doi:10.1016/j.asoc.2025.114213.

[31] L. Bergamin, M. Polato, F. Aiolli, Improving rule-based classifiers by Bayes point aggregation, Neurocomputing 613 (2025) 128699. doi:10.1016/j.neucom.2024.128699.

[32] D. H. Wolpert, Stacked generalization, Neural Networks 5 (2) (1992) 241–259.

[33] J. H. Friedman, Greedy function approximation: A gradient boosting machine, Annals of Statistics 29 (5) (2001) 1189–1232.

[34] R. Kohavi, A study of cross-validation and bootstrap for accuracy estimation and model selection, in: Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence (IJCAI), 1995, pp. 1137–1145.

[35] D. Dua, C. Graff, UCI machine learning repository, `https://archive.ics.uci.edu` (2019).

[36] J. Vanschoren, J. N. van Rijn, B. Bischl, L. Torgo, OpenML: Networked science in machine learning, ACM SIGKDD Explorations Newsletter 15 (2) (2014) 49–60.

[37] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine Learning Research 7 (2006) 1–30.

[38] L. I. Kuncheva, C. J. Whitaker, Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy, Machine Learning 51 (2) (2003) 181–207. `doi:10.1023/A:1022859003006`.

[39] D. D. Margineantu, T. G. Dietterich, Pruning adaptive boosting, in: Proceedings of the 14th International Conference on Machine Learning (ICML), Morgan Kaufmann, 1997, pp. 211–218.

[40] T. G. Dietterich, An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization, Machine Learning 40 (2) (2000) 139–157. `doi:10.1023/A:1007607513941`.